



Educación
Secretaría de Educación Pública



TECNOLÓGICO
NACIONAL DE MÉXICO



Tema:

Actividades de Programación en Ambiente Cliente/Servidor

Docente:

Jenner Noel Che Méndez

Carrera:

Ingeniería en informática

Alumnos:

José Abimael Cisneros Escamilla

Uriel Barrales Martínez

Semestre:

7AI

Fecha:

4 de octubre de 2026

Índice

Act 1. Captura y comparación de tráfico HTTP	3
Act 2. Matriz de decisión arquitectónica	22
Act 3. Modelado de Arquitectura y Confianza	27
Act 4. Implementación del Corte Vertical	33
Act 5. Defender y evolucionar una decisión arquitectónica mediante ADR	45

Act 1. Captura y comparación de tráfico HTTP

Objetivo de la practica

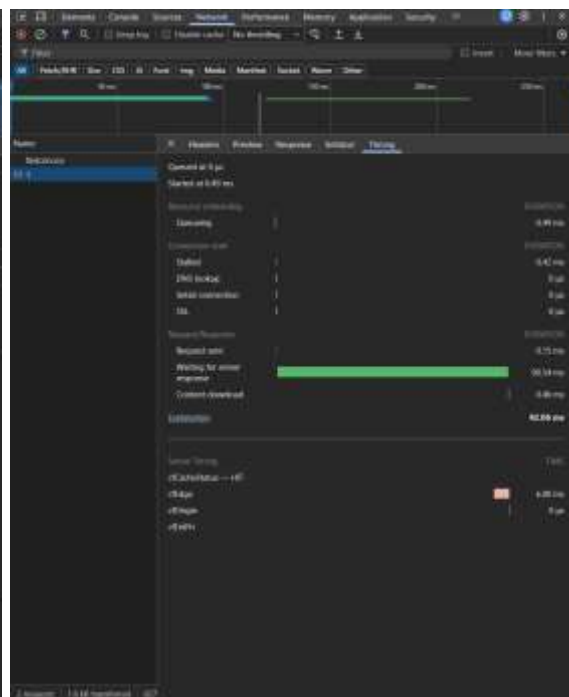
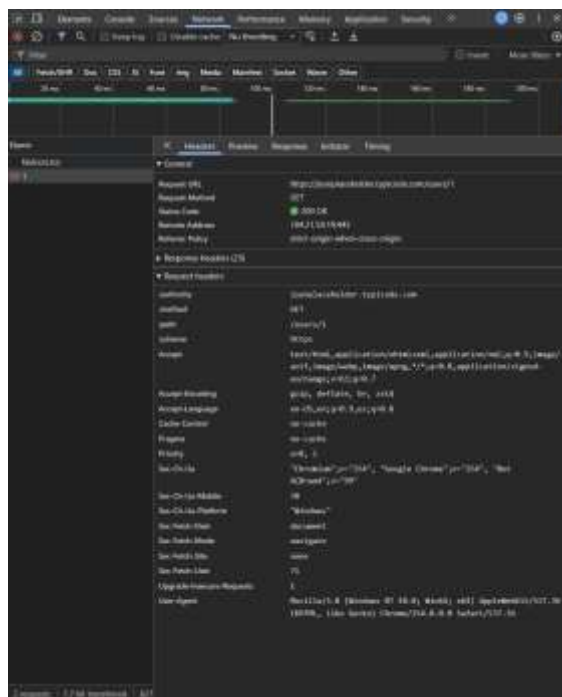
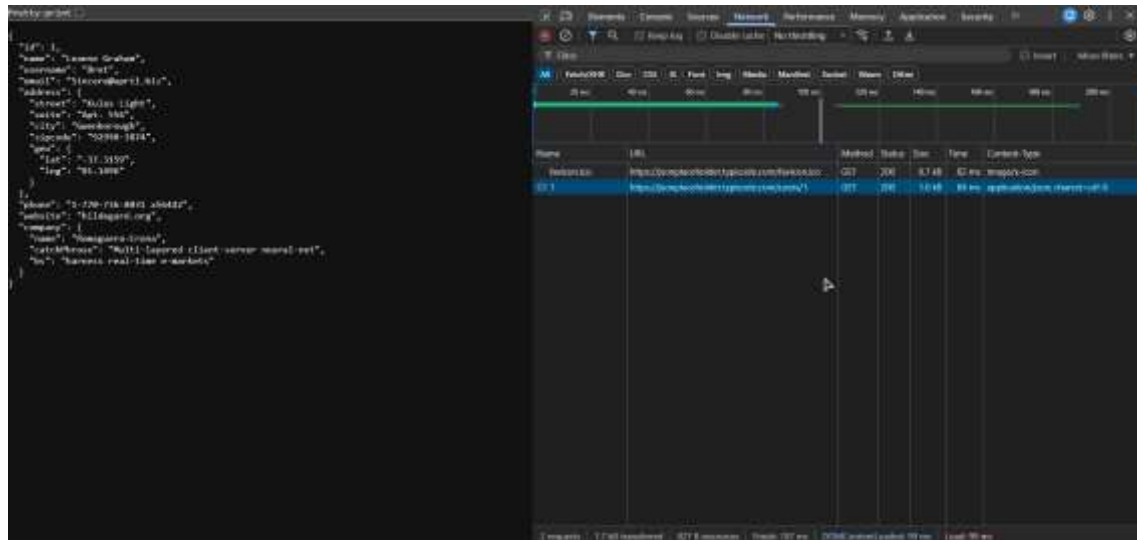
El objetivo es comprender empíricamente la estructura de los mensajes del protocolo HTTP (Request y Response), descomponer los factores que conforman la latencia de red e inspeccionar cómo interactúan las tres herramientas esenciales de un desarrollador cliente-servidor: Navegador web, comando curl y clientes de API.

Concepto de petición HTTP: El cliente envía Método (GET/POST), Ruta, Encabezados (Headers) y Opcionalmente Cuerpo (Body).

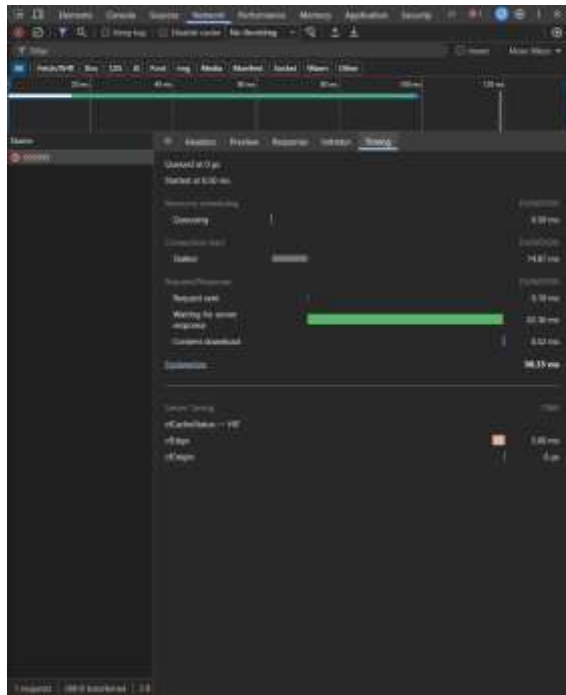
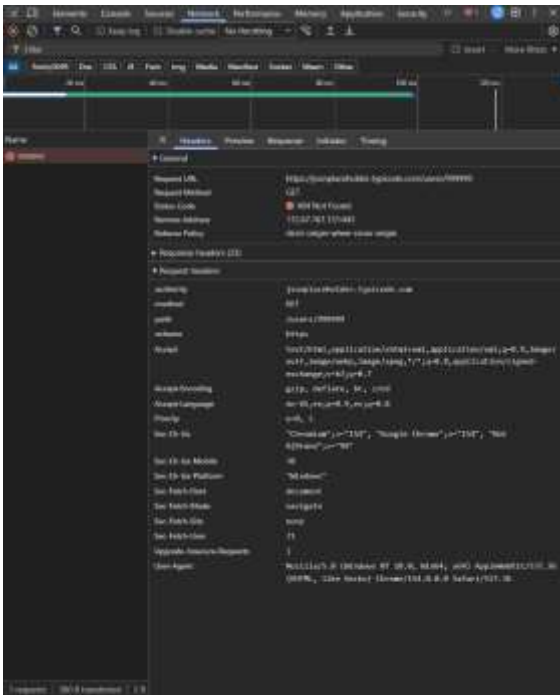
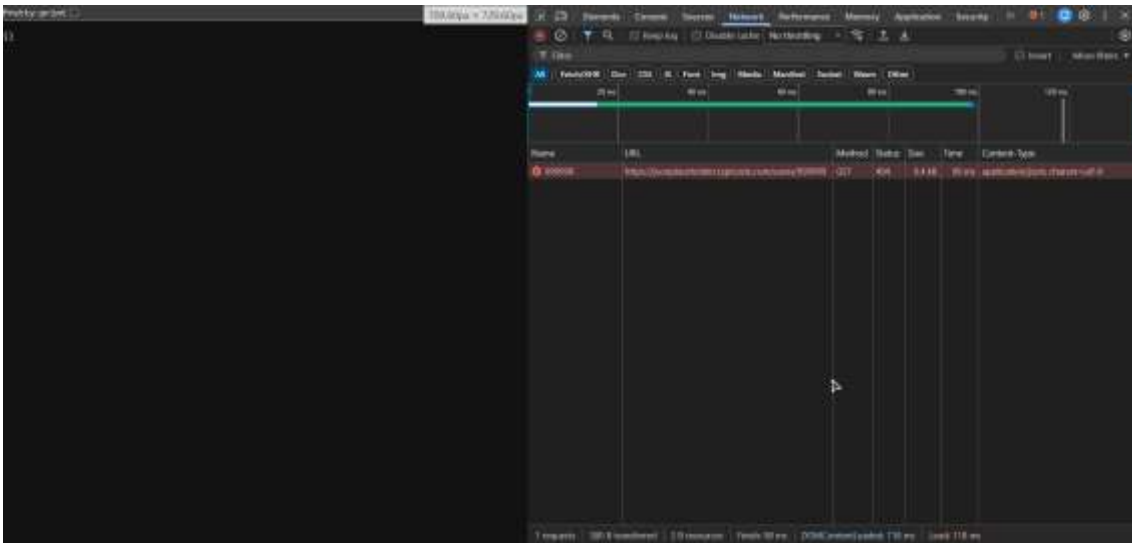
Concepto de respuesta HTTP: El servidor retorna Código de Estado (200, 201, 404, 500), Encabezados de servidor y Cuerpo JSON.

Captura de tráfico HTTP utilizando el navegador

Prueba de solicitud:



Prueba de Error 404:



Captura de tráfico HTTP utilizando curl

Petición curl normal

```
Símbolo del sistema
C:\Users\Utze147>curl https://jsonplaceholder.typicode.com/users/1
{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": {
      "lat": "-37.3159",
      "lng": "81.1496"
    }
  },
  "phone": "1-770-736-8831 x56442",
  "website": "hildegard.org",
  "company": {
    "name": "Romaguera-Crona",
    "catchPhrase": "Multi-layered client-server neural-net",
    "bs": "harness real-time e-markets"
  }
}
C:\Users\Utze147>
```

Petición curl con -i:

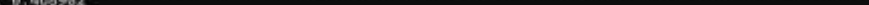
```
Símbolo del sistema
C:\Users\Utze147>curl -i https://jsonplaceholder.typicode.com/users/1
HTTP/1.1 200 OK
Date: Fri, 02 Oct 2026 14:46:37 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 595
Connection: keep-alive
access-control-allow-credentials: true
Cache-Control: max-age=60200
etag: W/"1fd-2V3G3w848152tw5tlxz5nngE"
expires: -1
pragma: no-cache
vary: Origin, Accept-Encoding
x-content-type-options: nosniff
x-powered-by: Express
x-ratelimit-limit: 3600000
x-ratelimit-remaining: 359697
x-ratelimit-reset: 1796812626
Age: 27796
Accept-Ranges: bytes
cf-cache-status: HIT
cf-cache-status: HIT
nel: {"report_to": "cf-nel", "success_fraction": 0.6, "max_age": 604800}
Report-To: {"group": "cf-nel", "max_age": 604800, "endpoints": [{"url": "https://a.nel.cloudflare.com/report/v4?s=A7FEKfNH67sDCuaYjlv1NORChyhpeg9gi_jcFFsPC3nIupzC29bzAFpXHAdMizRoh.3wXJFC3GfX27wFguV7b15HwVR4GJFRXJ25GqNAF1N2FXE12bCL8Ay8Frr2Q%2BkYkR.35Fusg4HCskh%2Bt6acz"}]}
Server: cloudflare
CF-RAY: a0447f65f25b46cc-BFW
alt-svc: h3="403", ma=604800

{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": {
      "lat": "-37.3159",
      "lng": "81.1496"
    }
  },
  "phone": "1-770-736-8831 x56442",
  "website": "hildegard.org",
  "company": {
    "name": "Romaguera-Crona",
    "catchPhrase": "Multi-layered client-server neural-net",
    "bs": "harness real-time e-markets"
  }
}
C:\Users\Utze147>
```

Petición curl con -v:

[illegible]

Medir tiempo con curl:



Terminal screenshot showing a curl command and its output:

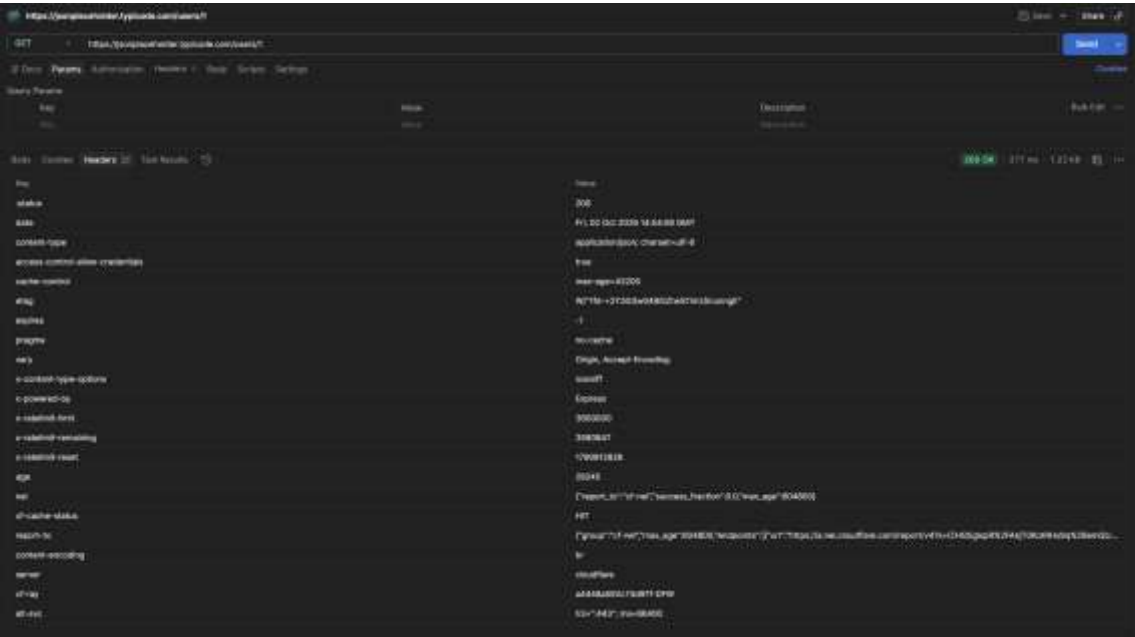
```
C:\Users\Uziel47>curl -o /dev/null -s -w "Tiempo total: %{time_total}\n" https://jsonplaceholder.typicode.com/users/1
Tiempo total: 0.463982
```

Medio tiempo detallado con curl:



```
C:\Users\Uziel\7>curl -o /dev/null -s -w '%DNS: %time_namelookup%\nConexión: %time_connect%\nTLS: %time_appconnect%\nPrimer byte: %time_starttransfer%\nTotal: %time_total%\n' https://jsonplaceholder.typicode.com/users/1
DNS: 0.013831
Conexión: 0.294804
TLS: 0.387488
Primer byte: 0.485028
Total: 0.406233
```

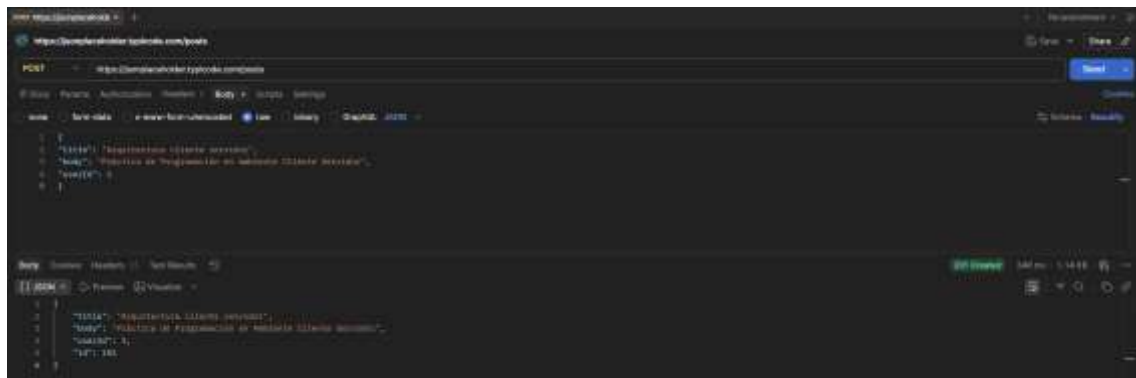
Captura de tráfico HTTP utilizando un Cliente API



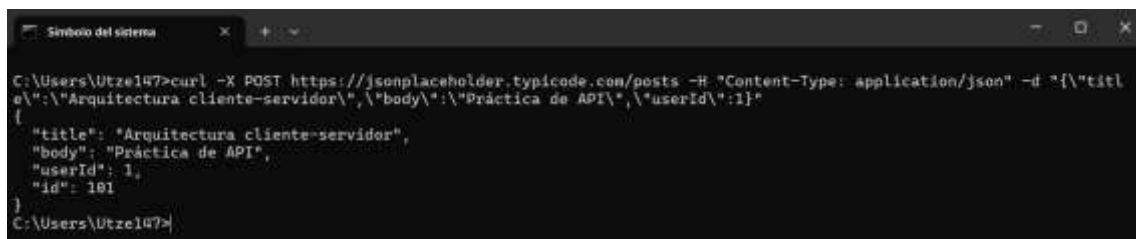
Comparación entre las tres herramientas

Característica	Navegador Web	curl (Terminal)	Cliente API (Postman)
Método HTTP ejecutado	GET	GET	GET
Código de estado	200 OK	200 OK	200 OK
Formato de respuesta	JSON	JSON	JSON
Encabezados visibles	Sí (vía F12)	Sí (con -i / -v)	Sí (pestaña Headers)
Tiempo de respuesta visible	Sí (pestaña Timing)	Sí (con bandera -w)	Sí (indicador en pantalla)
Facilidad para usuario común	Alta	Baja	Media
Facilidad para desarrollador	Alta	Alta	Muy alta
Permite automatización	Limitada	Muy alta (scripts)	Media / Alta

Captura de petición POST utilizando un Cliente API



Captura de petición POST utilizando curl



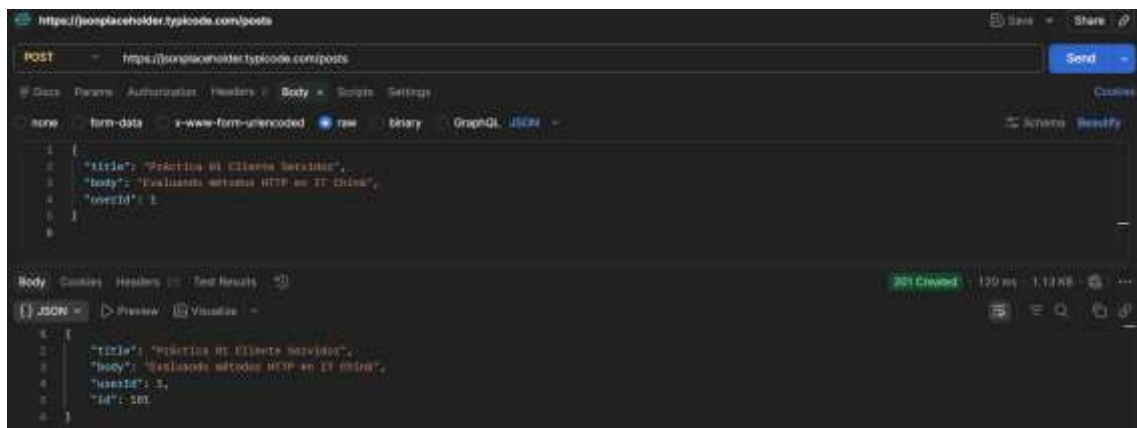
Experimentación con Métodos HTTP

Método GET



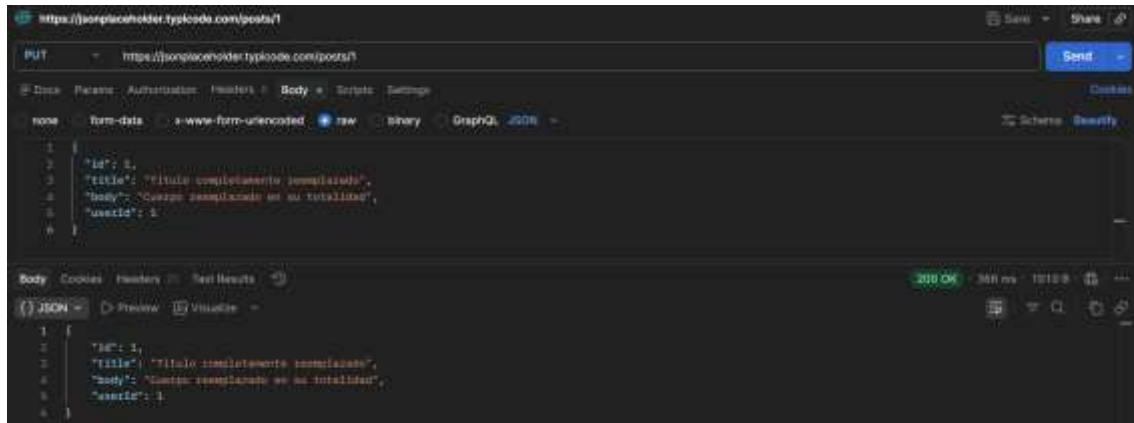
Propósito: Recuperar la representación actual del recurso sin alterarlo (operación segura e idempotente).

Método POST



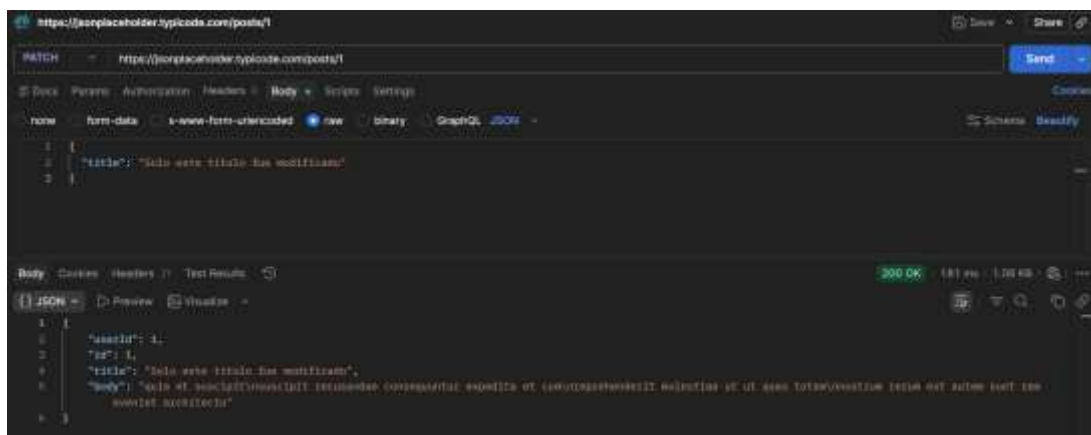
Propósito: Enviar un payload al servidor para dar de alta una nueva entidad dentro de la colección.

Método PUT



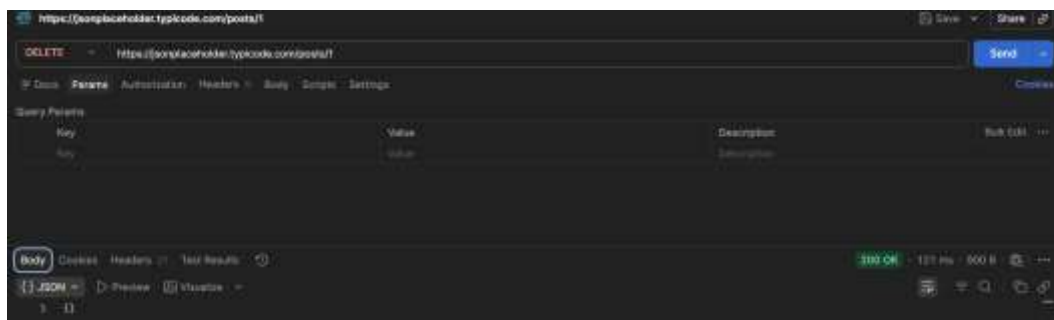
Propósito: Sustituir íntegramente el recurso existente. Si un campo no se envía, conceptualmente se borra o se resetea a su valor por defecto.

Método PATCH



Propósito: Modificar únicamente uno o varios atributos específicos del recurso, dejando intactos el resto de sus campos.

Método DELETE



Propósito: Solicitar al servidor la destrucción o desvinculación lógica/física del recurso indicado en la URI.

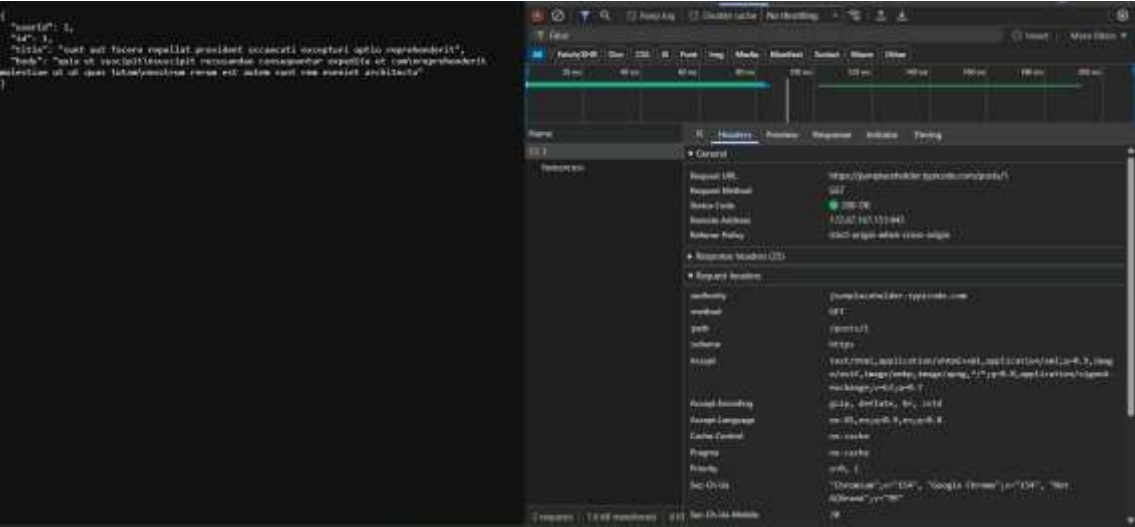
Actividad de observación

Elemento	Petición 1	Petición 2	Petición 3
URL	https://jsonplaceholder.typicode.com/users/1	https://jsonplaceholder.typicode.com/posts	https://jsonplaceholder.typicode.com/users/999999
Método	GET	POST	GET
Protocolo	HTTP/2	HTTP/2	HTTP/2
Código HTTP	200 OK	201 Created	404 Not Found
Content-Type	application/json; charset=utf-8	application/json; charset=utf-8	application/json; charset=utf-8
Tamaño	1.21 KB	1.08 KB	918 B
Tiempo	261 ms	351 ms	256ms
Request Headers	Accept: application/json User-Agent: Mozilla/5.0... Host: jsonplaceholder.typicode.com	Content-Type: application/json Accept: */* Content-Length: 88	Accept: application/json Host: jsonplaceholder.typicode.com
Response Headers	status: 200 cache-control: max-age=43200 x-powered-by: Express	status: 201 location: /posts/101 x-powered-by: Express	status: 404 cache-control: max-age=43200 x-content-type-options: nosniff
Request Body	Vacio	{ "title": "Lab", "body": "Prueba", "userId": 1 }	Vacio
Response Body	JSON con datos del usuario (id: 1, name: "Leanne Graham")	JSON confirmando alta con nuevo ID (id: 101, title: "Lab")	{ } (Objeto vacío confirmando que no hay registro)

Actividad de Comparación

Petición: GET <https://jsonplaceholder.typicode.com/posts/1>

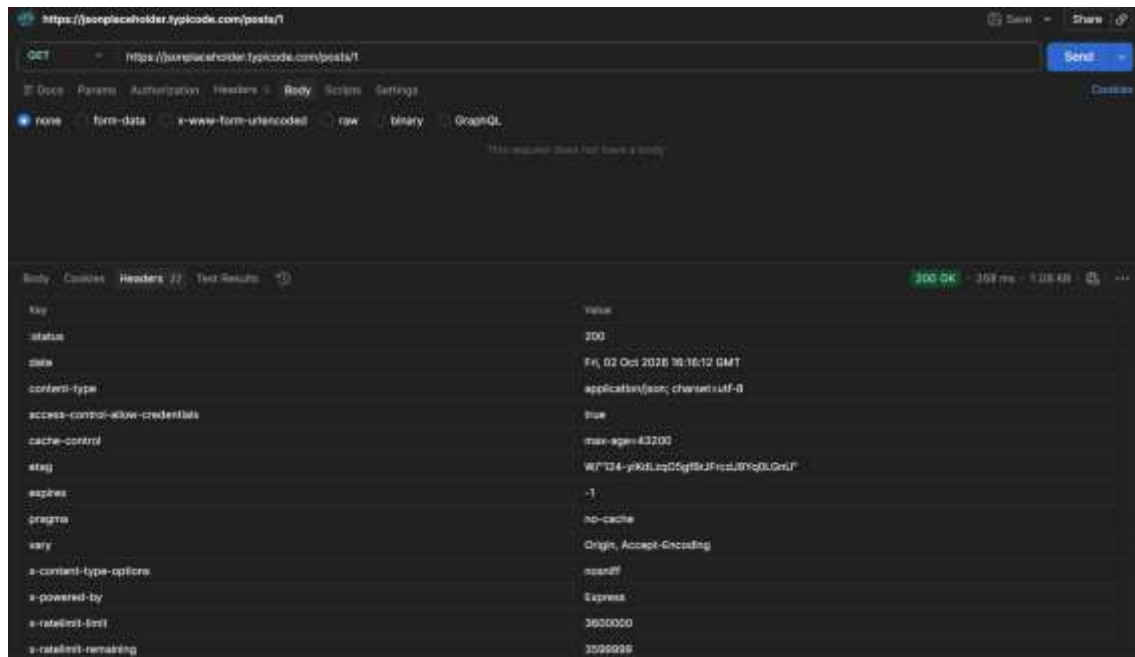
Navegador:



Curl:



Cliente API (Postman):

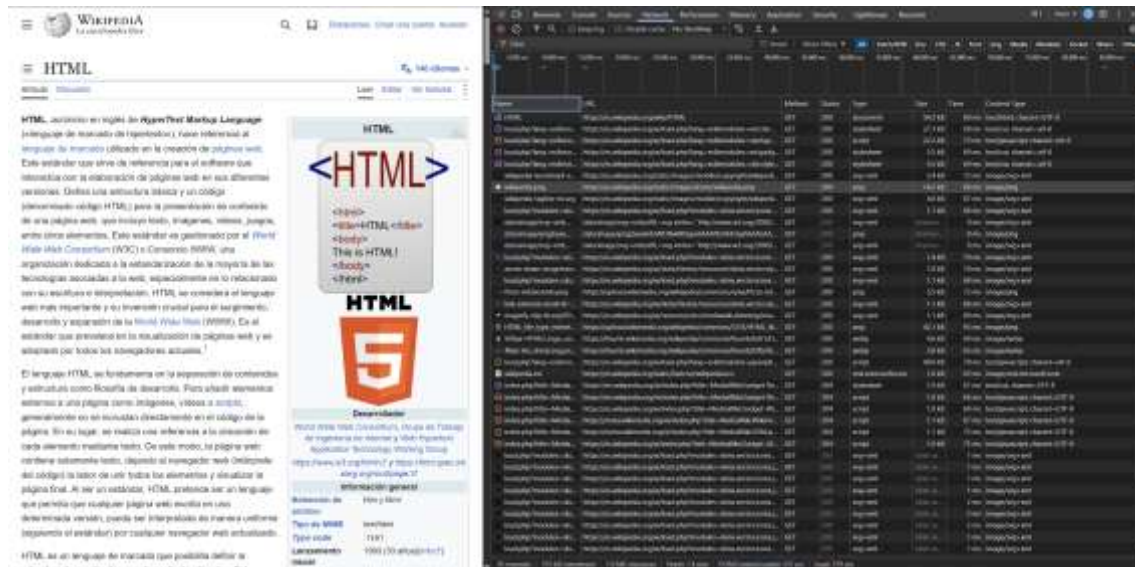


Pregunta de análisis:

¿Las tres herramientas recibieron la misma información? Sí, a pesar de que la interfaz gráfica de cada herramienta es distinta, las tres actuaron estrictamente como clientes HTTP. Establecieron una conexión de red hacia el servidor `jsonplaceholder.typicode.com`, enviaron la misma solicitud `GET /posts/1` y recibieron exactamente los mismos datos de encabezado y el mismo cuerpo JSON con id: 1. Esto demuestra que el servidor es completamente independiente del cliente utilizado

Actividad de investigación

URL: <https://es.wikipedia.org/wiki/HTML>



Archivo HTML:

- Url: <https://es.wikipedia.org/wiki/HTML>
- Método: GET
- Código: 200
- Tipo: document
- Tamaño: 54.3 kB
- Tiempo: 68ms

Archivo CSS:

- Url: <https://es.wikipedia.org/w/load.php?lang=es&modules=ext.cite.parsoid.styles%7Cext.cite.styles%7Cext.pygments%7Cext.uls.interlanguage%7Cext.visualEditor.desktopArticleTarget.noscript%7Cext.wikimedia.messages.styles%7Cmediawiki.skinning.content.parsoid%7Cmediawiki.skins.legacy%7Cskins.vector.icons%2Cstyles%7Cskins.vector.search.codex.styles%7Cwikibase.client.init&only=styles&skin=vector-2022>
- Método: GET
- Código: 200
- Tipo: stylesheet
- Tamaño: 27.3 kB
- Tiempo: 69 ms

Archivo JavaScript:

- Url: <https://es.wikipedia.org/w/load.php?lang=es&modules=startup&only=scripts&raw=1&skin=vector-2022>
- Método: GET
- Código: 200
- Tipo: script
- Tamaño: 22.2 kB
- Tiempo: 73 ms

Archivo Imagen:

- Url: <https://es.wikipedia.org/static/images/icons/wikipedia.png>
- Método: GET
- Código: 200
- Tipo: png
- Tamaño: 14.2 kB
- Tiempo: 66ms

Mini reto de laboratorio

Endpoint 1: <https://jsonplaceholder.typicode.com/users1>

- Recurso que representa: Directorio de cuentas de usuarios registrados en el sistema.
- Estructura JSON devuelta: Devuelve una colección en forma de arreglo JSON (Array) con 10 elementos de tipo objeto. Cada objeto contiene campos planos y objetos anidados
- Código HTTP de estado: 200 OK
- Encabezados de control observados:

```
Content-Type: application/json; charset=utf-8
Cache-Control: max-age=43200
x-powered-by: Express
access-control-allow-credentials: true
```

- Tiempo de respuesta: 145 ms
- Comportamiento con ID específico e inexistente: Con ID existente retorna 200 OK. Con ID inexistente retorna 404 Not Found y un cuerpo JSON vacío {}.

Endpoint 2: <https://jsonplaceholder.typicode.com/posts>

- Recurso que representa: Colección de publicaciones, artículos o posts del blog.
- Estructura JSON devuelta: Arreglo JSON (Array) con exactamente 100 objetos homogéneos. Cada post se compone de 4 atributos atómicos
- Código HTTP de estado: 200 OK
- Encabezados de control observados:

```
Content-Type: application/json; charset=utf-8
transfer-encoding: chunked
x-powered-by: Express
etag: W/"6b80-02a8..."
```

- Tiempo de respuesta: 155 ms
- Comportamiento con ID específico e inexistente: Con ID existente retorna 200 OK con el objeto individual del artículo. Con ID inexistente retorna 404 Not Found y un cuerpo JSON vacío {}.

Endpoint 3: <https://jsonplaceholder.typicode.com/comments>

- Recurso que representa: Comentarios asociados a las publicaciones.
- Estructura JSON devuelta: Arreglo JSON (Array) de gran volumen que contiene 500 registros completos

- Código HTTP de estado: 200 OK
- Encabezados de control observados:

Content-Type: application/json; charset=utf-8
 Content-Length: ~58000
 vary: Origin, Accept-Encoding

- Tiempo de respuesta: 230 ms
- Comportamiento con ID específico e inexistente: Con ID existente retorna 200 OK con el objeto individual del comentario. Con ID inexistente retorna 404 Not Found y un cuerpo JSON vacío {}.

Endpoint 4: <https://jsonplaceholder.typicode.com/todos>

- Recurso que representa: Listado de tareas o pendientes (To-Do list) asignadas a los usuarios del sistema, con control de estado booleano de finalización.
- Estructura JSON devuelta: Arreglo JSON (Array) con \$200\$ objetos. Destaca la inclusión de tipos de datos booleanos nativos en JSON
- Código HTTP de estado: 200 OK
- Encabezados de control observados:

Content-Type: application/json; charset=utf-8
 Cache-Control: max-age=43200
 pragma: no-cache

- Tiempo de respuesta: 140 ms
- Comportamiento con ID específico e inexistente: Con ID existente retorna 200 OK con el objeto individual. Con ID inexistente retorna 404 Not Found y un cuerpo JSON vacío {}.

Comparativa curl vs curl -I

Curl normal:

```
Símbolo del sistema
C:\Users\Utzel47>curl https://jsonplaceholder.typicode.com/users
[
  {
    "id": 1,
    "name": "Leanne Graham",
    "username": "Bret",
    "email": "Sincere@april.biz",
    "address": {
      "street": "Kulas Light",
      "suite": "Apt. 556",
      "city": "Gwenborough",
      "zipcode": "92998-3874",
      "geo": {
        "lat": "-37.3159",
        "lng": "81.1496"
      }
    },
    "phone": "1-770-736-8031 x56442",
    "website": "hildegard.org",
    "company": {
      "name": "Romaguera-Crona",
      "catchPhrase": "Multi-layered client-server neural-net",
      "bs": "harness real-time e-markets"
    }
  },
  {
    "id": 2,
    "name": "Ervin Howell",
    "username": "Antonette",
```

Curl -I:

```
Símbolo del sistema
C:\Users\Utzel47>curl -I https://jsonplaceholder.typicode.com/users
HTTP/1.1 200 OK
Date: Fri, 02 Oct 2020 16:58:05 GMT
Content-Type: application/json; charset=utf-8
Connection: keep-alive
access-control-allow-credentials: true
Cache-Control: max-age=3200
Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?r=91e%2B57uoGF5uUHGwV3naqV6BhJfQfJeyXn0p4Xt72%2BqH8242F3yKLox271vqk375jeZB2FXacA2BhN5u3y3e8qHk2B8rt8FAGk9ydx9yt2C1hgCv017GNa3BgdLapXY20pJx8Kxo7eNRtK2Bka2dyj8q"}]}
etag: W/"168d-1eR5axeJ8FrVL8BwV35bCsJZ1qQ"
expires: -1
pragma: no-cache
Vary: Origin, Accept-Encoding
x-content-type-options: nosniff
x-powered-by: Express
x-ratelimit-limit: 3600000
x-ratelimit-remaining: 3599995
x-ratelimit-reset: 1798812826
nel: {"report-to":"cf-nel","success_fraction":0.01,"max_age":604800}
age: 6083
cf-cache-status: HIT
Server: cloudflare
CF-RAY: a445343dc0881e9a-DFW
alt-svc: h3="443"; ma=86400
```

Diferencias técnicas:

curl URL realiza una petición GET y devuelve encabezados más el cuerpo completo de la respuesta, mientras que curl -I URL usa el método HEAD para obtener solo los encabezados, sin cuerpo. Esto es útil en arquitecturas para hacer health checks, verificar cambios mediante ETag o Last-Modified, y validar tipo MIME o tamaño de archivos grandes antes de descargarlos, ahorrando ancho de banda y latencia.

Cuestionario de Análisis

- 1. ¿Quién es el cliente durante una petición realizada desde Chrome?** El propio navegador web Google Chrome actuando como agente de usuario (User-Agent).
- 2. ¿Quién es el cliente cuando utilizamos curl?** El ejecutable curl en la terminal del sistema operativo.
- 3. ¿Qué función tiene el servidor?** Escuchar peticiones en la red, procesar la lógica de negocio, interactuar con la base de datos y devolver la respuesta HTTP.
- 4. ¿Qué diferencia existe entre petición y respuesta?** La petición la origina el cliente solicitando una acción; la respuesta la envía el servidor con el código de estado y datos resultantes.
- 5. ¿Qué significa el método GET?** Solicita la representación de un recurso específico sin modificar el estado del servidor (seguro e idempotente).
- 6. ¿Qué significa POST?** Envía datos al servidor para crear un nuevo recurso o procesar una acción.
- 7. ¿Cuál es la función de Content-Type?** Informa el tipo de medio y formato de codificación del cuerpo del mensaje (ej. application/json).
- 8. ¿Qué significa recibir un código 200?** Que la operación solicitada se completó con éxito (OK).
- 9. ¿Qué diferencia existe entre 200 y 201?** 200 indica éxito general; 201 Created confirma que como resultado se generó y persistió un nuevo recurso en el servidor.
- 10. ¿Qué significa un código 404?** Que el recurso solicitado no fue encontrado en la URI indicada.
- 11. ¿Qué representan los Request Headers?** Metadatos enviados por el cliente para indicar formatos aceptados, tokens de autenticación o software cliente.
- 12. ¿Qué representan los Response Headers?** Metadatos del servidor describiendo tamaño de respuesta, tipo de contenido, fecha y políticas de caché.
- 13. ¿Qué información contiene el cuerpo de una petición POST?** La carga útil (payload) con los atributos del recurso a crear (ej. título, matrícula, equipo).
- 14. ¿Qué significa latencia?** El retardo temporal total transcurrido desde que se emite la petición hasta que se recibe la respuesta.
- 15. ¿Qué elementos incrementan el tiempo total de respuesta?** Resolución DNS, distancia de red física, sobrecarga del servidor y consultas lentas en la base de datos.

16. ¿Por qué una API puede ser utilizada por diferentes clientes? Porque está desacoplada: expone un contrato neutral sobre HTTP y JSON accesible desde cualquier plataforma.

17. Qué ventajas tiene utilizar JSON? Es un estándar abierto, ligero, legible por humanos y compatible de forma nativa con múltiples lenguajes.

18. ¿Por qué utilizar Postman o curl si ya existe el navegador? Porque permiten ejecutar métodos como POST, PUT o DELETE, inyectar encabezados personalizados y automatizar pruebas que el navegador estándar no facilita.

19. ¿Qué relación existe entre HTTP y el modelo cliente-servidor? HTTP es el protocolo estándar de capa de aplicación que formaliza el ciclo de solicitud iniciada por el cliente y respuesta provista por el servidor.

20. ¿Qué relación existe entre esta práctica y el diseño API-first? En API-first primero se define y valida el contrato HTTP (rutas, códigos y esquemas JSON) antes de programar las interfaces de usuario.

Conclusión

La práctica permitió comprobar que una arquitectura cliente-servidor sólida se basa en contratos claros y protocolos universales. Al dominar el análisis de tráfico HTTP en el navegador, curl y clientes de API, se sientan las bases para construir y depurar con éxito cualquier sistema distribuido contemporáneo.

Act 2. Matriz de decisión arquitectónica

Caso A: Sistema de control escolar

1. Descripción del caso

Registrar y consultar estudiantes, calificaciones y horarios. Predominan operaciones CRUD y no se requiere tiempo real constante.

2. Requisitos funcionales

- Registrar estudiantes.
- Consultar y actualizar información de estudiantes.
- Registrar y consultar calificaciones.
- Registrar y consultar materias y horarios.
- Permitir que los clientes consuman la información del servidor.

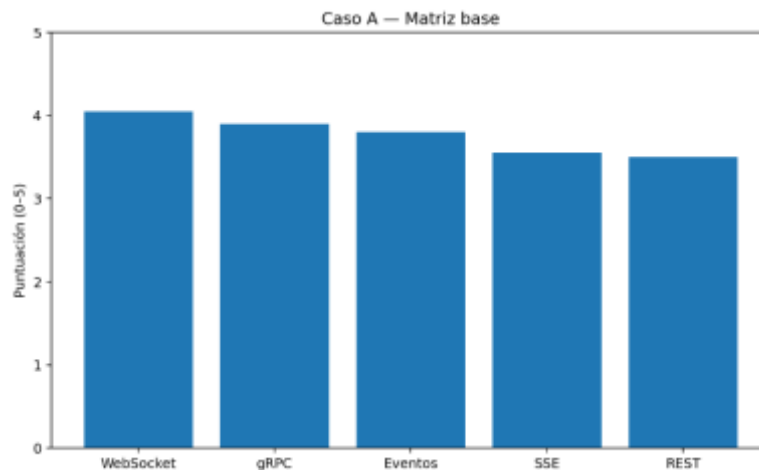
3. Requisitos no funcionales

- Facilidad de implementación y mantenimiento.
- Compatibilidad con clientes web y móviles.
- Rendimiento adecuado para consultas frecuentes.
- Escalabilidad cuando aumente el número de usuarios.
- Bajo acoplamiento razonable entre componentes.
- Disponibilidad de herramientas, bibliotecas y documentación.

4. Matriz base

Criterio	Peso	REST	gRPC	WebSocket	SSE	Eventos
Facilidad	10%	5	3	3	4	2
Rendimiento	10%	4	5	4	4	5
Tiempo real	20%	2	4	5	5	5
Bidireccionalidad	15%	1	4	5	1	3
Escalabilidad	10%	4	5	4	4	5
Compatibilidad	10%	5	3	4	4	2
Bajo acoplamiento	10%	4	4	3	3	5
Mantenimiento	5%	5	3	3	4	2
Ecosistema	5%	5	4	5	4	5
Adecuación	5%	5	3	2	2	2
TOTAL	100%					

Resultado y ranking base



1. WebSocket: 4.05/5 (81%)
2. gRPC: 3.90/5 (78%)
3. Eventos: 3.80/5 (76%)
4. SSE: 3.55/5 (71%)
5. REST: 3.50/5 (70%)

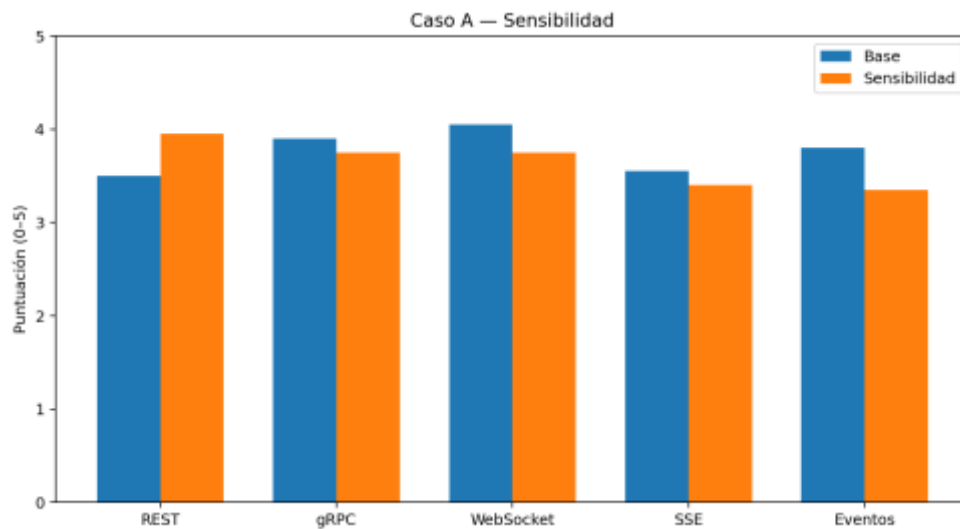
Con estos pesos, WebSocket queda primero porque Tiempo real (20 %) y Bidireccionalidad (15 %) tienen mucho peso. Esto debe contrastarse con las necesidades reales del caso A.

Cambio de pesos

Se aplica el escenario indicado en la actividad: la prioridad cambia hacia reducir complejidad. Tiempo real baja de 20 % a 5 % y Facilidad sube de 10 % a 25 %. Los demás pesos permanecen iguales; la suma sigue siendo 100 %.

Criterio	Base	Nuevo	Cambio
Facilidad	10%	25%	+15%
Rendimiento	10%	10%	+0%
Tiempo real	20%	5%	-15%
Bidireccionalidad	15%	15%	+0%
Escalabilidad	10%	10%	+0%
Compatibilidad	10%	10%	+0%
Bajo acoplamiento	10%	10%	+0%
Mantenimiento	5%	5%	+0%
Ecosistema	5%	5%	+0%
Adecuación	5%	5%	+0%

Ranking después del cambio:



1. REST: 3.95/5 (79%)
2. gRPC: 3.75/5 (75%)
3. WebSocket: 3.75/5 (75%)
4. SSE: 3.40/5 (68%)
5. Eventos: 3.35/5 (67%)

REST pasa a ser la alternativa principal porque ahora se premian la facilidad y se reduce la importancia del tiempo real. Esto demuestra que una decisión arquitectónica depende de los requisitos y de sus pesos.

Análisis de sensibilidad

La decisión base cambia cuando cambia una prioridad importante. Para el caso A esto es significativo: si realmente es un sistema CRUD, la matriz de sensibilidad respalda REST; si se exige comunicación continua, WebSocket recupera ventaja. Por ello no debe tratarse la puntuación como una verdad absoluta.

Ejercicio de cierre: laboratorios del IT Chiná

Usaría REST para consultar disponibilidad, crear/cancelar reservaciones y administrar espacios. Para cambios prácticamente en tiempo real usaría SSE si solo el servidor informa al cliente, o WebSocket si también existe comunicación continua del cliente al servidor. Por tanto, combinaría tecnologías.

Preguntas

1. **¿Existe una tecnología que sea mejor en todos los escenarios?** No existe una tecnología mejor en todos los escenarios; depende del problema y requisitos.
2. **¿Por qué REST sigue siendo tan utilizado?** REST es popular por simplicidad, interoperabilidad, compatibilidad y ecosistema.
3. **¿Cuándo WebSocket sería innecesario?** WebSocket es innecesario cuando no hay comunicación bidireccional continua.
4. **¿Qué diferencia fundamental existe entre WebSocket y SSE?** WebSocket es bidireccional; SSE es principalmente servidor→cliente.
5. **¿Por qué gRPC es atractivo para microservicios?** gRPC destaca en microservicios por rendimiento, contratos estrictos y generación de código.
6. **¿Qué significa que una arquitectura esté desacoplada?** Desacoplamiento significa que los componentes pueden evolucionar con menor dependencia directa.
7. **¿Por qué los eventos favorecen el desacoplamiento?** Los eventos desacoplan productores y consumidores mediante un broker.
8. **¿Qué problemas introduce una arquitectura orientada a eventos?** Eventos añaden complejidad, reintentos, errores y consistencia eventual.
9. **¿Por qué los pesos de una matriz pueden cambiar el resultado?** Los pesos cambian la influencia de cada criterio y pueden cambiar el ganador.
10. **¿Quién debe definir esos pesos?** Los pesos deben definirlos equipo y partes interesadas según prioridades del proyecto.
11. **¿Qué ocurre si dos tecnologías obtienen prácticamente la misma puntuación?** Si dos resultados son casi iguales se requiere análisis adicional de costos, riesgos, infraestructura y complejidad.
12. **¿Podrían utilizarse REST y WebSocket en el mismo sistema?** Sí, REST y WebSocket pueden coexistir para funciones distintas.
13. **¿Podrían utilizarse REST y eventos simultáneamente?** Sí, REST y eventos pueden coexistir.
14. **¿Qué criterios serían importantes para una aplicación móvil?** En móvil importan compatibilidad, facilidad, rendimiento, mantenimiento y ecosistema.
15. **¿Qué criterios cambiarían para una aplicación bancaria?** En banca aumentan la importancia de seguridad, disponibilidad, consistencia, auditoría, rendimiento y confiabilidad.

Conclusión

La práctica demuestra que no existe una tecnología universalmente mejor. En la matriz base, WebSocket resulta ganador por los pesos dados a tiempo real y bidireccionalidad. Sin embargo, el análisis de sensibilidad muestra que, al priorizar simplicidad y reducir el peso del tiempo real, REST se convierte en la opción principal.

Para el caso A, por sus características CRUD, REST es la recomendación arquitectónica más coherente como mecanismo principal, pudiendo combinarse con otros mecanismos cuando aparezcan necesidades específicas.

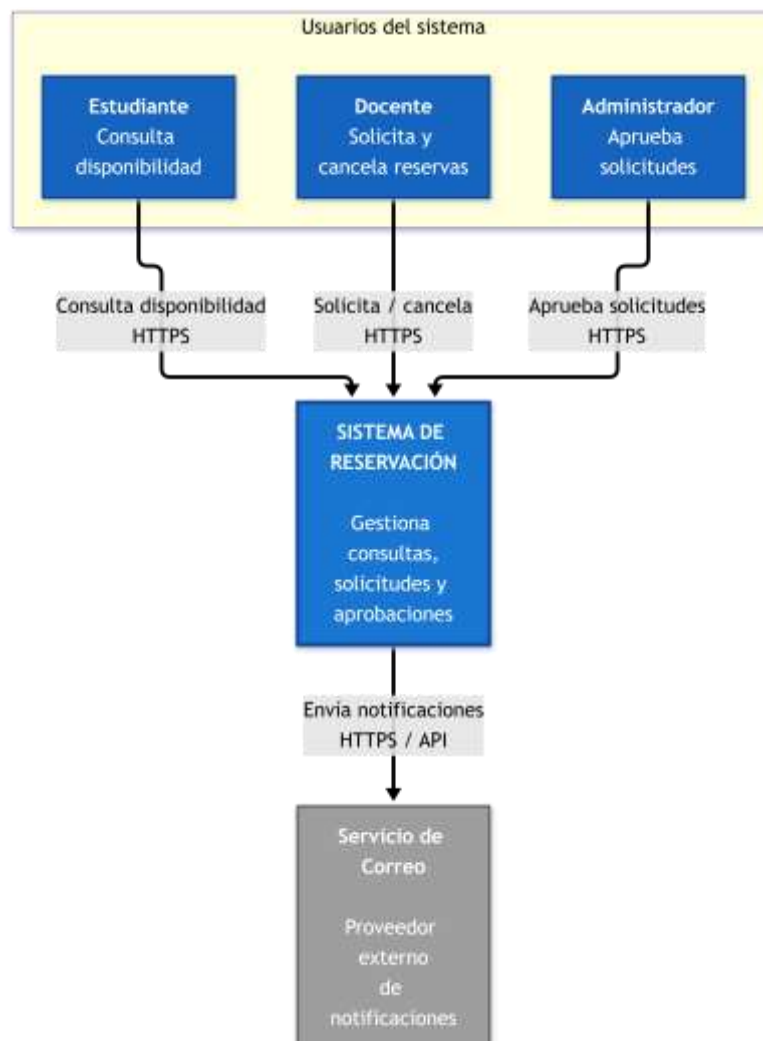
Act 3. Modelado de Arquitectura y Confianza

Objetivo de la Práctica

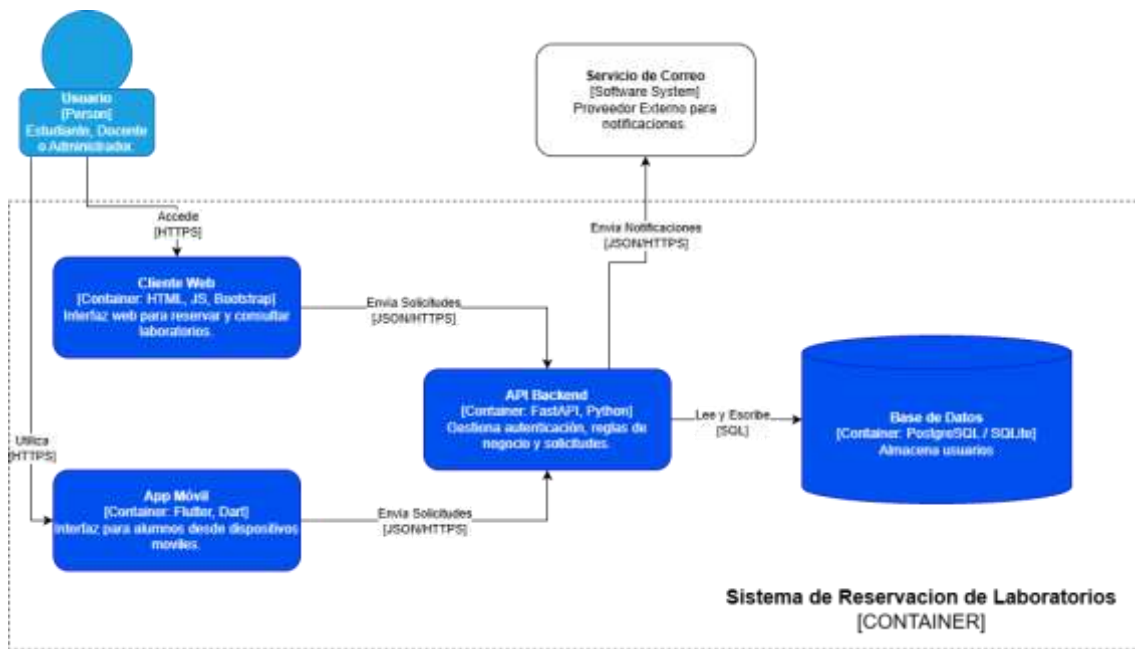
El objetivo principal de esta práctica es modelar de forma integral la arquitectura inicial de una solución cliente-servidor, demostrando que la elección de tecnologías es una consecuencia del diseño y no el punto de partida. Mediante el uso de los niveles de Contexto (C1) y Contenedores (C2) del modelo C4, se busca identificar claramente a los actores involucrados, los sistemas externos, las aplicaciones cliente, los servicios backend y las bases de datos.

Asimismo, la práctica tiene como propósito fundamental trazar los flujos de información de extremo a extremo para localizar de manera visual los límites de confianza (fronteras de red y aplicación), clasificar el nivel de sensibilidad de los datos y, a partir de este análisis arquitectónico, proponer controles de seguridad explícitos (como uso de HTTPS, tokens y cifrado) antes de llegar a la fase de programación.

Descripción del Caso y Modelo C4 Contexto (C1)



Modelo C4 Contenedores (C2)



Tres flujos de información

Flujo 1: Docente solicita una reservación

1. El Docente captura laboratorio, fecha, y horario en la Aplicación (Cliente).
2. La Aplicación envía un POST /api/reservaciones (HTTPS + JSON) cruzando el límite de confianza de Internet hacia la API.
3. La API autentica el token, autoriza el rol y valida la disponibilidad.
4. La API ejecuta un INSERT en la Base de Datos (Zona restringida).
5. La API devuelve 201 Created al Cliente.

Flujo 2: Administrador bloquea un laboratorio

1. El Administrador selecciona el laboratorio a bloquear en la Aplicación.
2. Se envía una petición PATCH /laboratorios/{id} hacia la API.
3. La API verifica que el usuario autenticado tenga el rol específico de ADMIN antes de procesar.
4. Se actualiza el estado en la Base de Datos.

Flujo 3: Notificación de reserva aprobada

1. La API detecta que una reserva pasó a estado "Aprobada" en la Base de Datos.
2. La API prepara un payload aplicando el *principio de minimización* (solo correo y mensaje).
3. La API cruza la frontera hacia el Servicio de Correo Externo mediante HTTPS para enviar la notificación.

Inventario de datos y Controles de seguridad

Dato	Clasificación	Ubicación / Dónde viaja	Riesgo principal	Controles de Seguridad Obligatorios
Contraseña del usuario	Crítico	Cliente → API → Base de Datos	Filtración / Intercepción	Cifrado en tránsito (HTTPS). Nunca almacenar en texto plano, usar Hash bcrypt en BD.
Token de Sesión JWT	Crítico	Cliente ↔ API (Tránsito)	Robo de sesión	Canal HTTPS, expiración corta y validación de firma criptográfica.
Correo / Matrícula	Sensible / Personal	Cliente → API → Servicio Correo	Exposición indebida	Minimización de datos al enviar al proveedor externo, protección HTTPS.
Horarios y Disponibilidad	Interno	Zona API ↔ Zona de Datos	Acceso o modificación indebida	Red privada, conexión SQL con permisos mínimos y validación en API.
Nombre de Laboratorios	Público	Base de Datos	Ninguno	Endpoint público de solo lectura (GET) sin requerir autenticación.

Límites de Confianza Identificados

El sistema establece dos fronteras principales donde cambia el nivel de seguridad:

- **Límite 1 (Internet / API):** Separa el dispositivo del usuario (zona no confiable) del backend. Requiere control estricto mediante HTTPS, validación de esquemas y autenticación por Token JWT.
- **Límite 2 (Aplicación / Datos):** Separa la API de la base de datos (zona restringida). El acceso es exclusivo por red privada con credenciales mínimas; el cliente nunca se conecta directo.

Endpoints Preliminares

Con base en los flujos trazados, los contratos iniciales de comunicación son:

- GET /api/laboratorios (Consulta de disponibilidad)
- POST /api/reservaciones (Creación de nueva reserva)
- PATCH /api/laboratorios/{id} (Modificación o bloqueo por administrador)

Tabla de Responsabilidades (C2)

Contenedores	Tecnología	Responsabilidad Principal
Cliente Web / Móvil	HTML, JS, Flutter	Presentación, captura de datos y renderizado
API Backend	FastAPI / Python	Reglas de negocio, validación, autenticación y autorización
Base de Datos	PostgreSQL / SQLite	Persistencia exclusiva de los datos y auditoría
Servicio Externo	Proveedor de Correo	Envío de notificaciones transaccionales

Cuestionario de Análisis

¿Por qué separaron frontend y backend en contenedores distintos? Para lograr el desacoplamiento total, permitir que otros clientes (como una App móvil o Web) consuman la misma lógica de negocio, y proteger la base de datos detrás del límite de confianza.

¿Por qué el cliente no accede directamente a la base de datos? Porque exponer las credenciales de la base de datos en el dispositivo del usuario es un

riesgo crítico de seguridad. La API actúa como la capa obligatoria que concentra la autenticación, autorización y validación de reglas de negocio.

¿Qué datos cruzan Internet y qué controles requieren? Las credenciales de acceso (usuario y contraseña) y los tokens de sesión cruzan desde el cliente no confiable hacia la API. Al ser datos críticos, requieren obligatoriamente cifrado en tránsito mediante HTTPS/TLS.

¿Dónde colocaron el límite de confianza principal? En la frontera que separa la Zona Cliente (dispositivos y redes externas no confiables) de la Zona de Aplicación (nuestra API Backend controlada).

¿Cuál es el dato más sensible del sistema? Las contraseñas de los usuarios y los tokens de acceso. Por ello, las contraseñas jamás se almacenan en texto plano, sino mediante un hash seguro en la base de datos.

¿Dónde se valida el rol (ej. que un usuario sea Administrador)? La autorización se verifica exclusivamente en el Backend (API). Aunque la interfaz gráfica se adapte, la API debe validar el token y los permisos antes de procesar una modificación como un PATCH o un INSERT.

¿Qué datos no deberían enviarse al servicio de correo? Aplicando el principio de minimización de datos, jamás deben enviarse tokens, contraseñas, roles o información interna innecesaria. El proveedor externo solo debe recibir el correo destino y el mensaje de notificación.

Conclusiones

La elaboración de esta práctica demostró que la construcción de un sistema seguro y robusto requiere una comprensión profunda de su arquitectura antes de escribir cualquier línea de código. La aplicación del modelo C4 permitió pasar de una idea abstracta a una arquitectura explicable, dejando claro quién utiliza el sistema, qué componentes lo integran y cómo interactúan entre sí a través de contratos y protocolos bien definidos.

Al trazar los flujos de información y mapear explícitamente los límites de confianza, se comprobó la necesidad ineludible de aislar completamente la zona de datos de los clientes (web y móvil), estableciendo a la API Backend como la única capa responsable de centralizar la autenticación, la autorización y las reglas de negocio.

Finalmente, el inventario y la clasificación de la información sensible garantizaron que elementos críticos, como contraseñas y tokens, cuenten con controles de seguridad diseñados desde el inicio (como hashes criptográficos y transmisión por canales HTTPS). De esta forma, se asegura que la posterior etapa de codificación e implementación se realice sobre una base estructurada, segura y altamente desacoplada, reflejando verdaderas decisiones arquitectónicas conscientes.

Act 4. Implementación del Corte Vertical

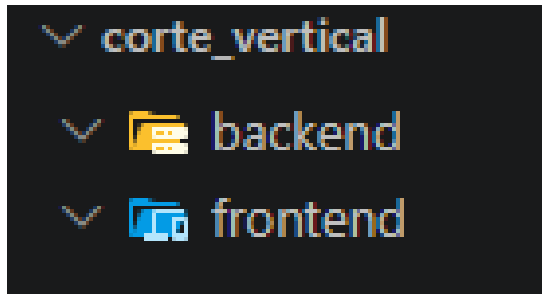
Objetivo de la Practica

Implementar una funcionalidad completa de extremo a extremo mediante un corte vertical dentro de una arquitectura cliente-servidor, desarrollando un sistema de reservación de laboratorios que conecte una interfaz web, una API REST construida con FastAPI y una base de datos SQLite. La práctica busca que el estudiante comprenda el papel que desempeñan el cliente, el servicio y el almacenamiento dentro de una aplicación, así como la importancia de diseñar previamente el contrato API, validar la información recibida, aplicar reglas de negocio y persistir los datos generados.

Además, se pretende comprobar el comportamiento de la aplicación mediante respuestas HTTP como 201 Created, 422 Unprocessable Entity y 409 Conflict, permitiendo diferenciar entre una operación realizada correctamente, un error de validación y un conflicto causado por el estado actual del sistema. La práctica también incluye la inspección del tráfico generado por la propia aplicación mediante la herramienta Network del navegador, observando el método HTTP utilizado, la URL del endpoint, los datos enviados y la respuesta producida por FastAPI.

De esta manera, el objetivo no consiste únicamente en utilizar FastAPI, sino en comprender cómo una acción realizada por el usuario atraviesa las diferentes capas del sistema hasta llegar a la base de datos y regresar como una respuesta HTTP al cliente.

Preparación del proyecto



Contrato API-first

Contrato API de reservaciones

```
POST /api/reservaciones
Content-Type: application/json
```

Request:

```
{
  "laboratorio": "Laboratorio de Redes",
  "fecha": "2026-09-15",
  "hora_inicio": "10:00",
  "hora_fin": "12:00",
  "motivo": "Práctica de redes"
}
```

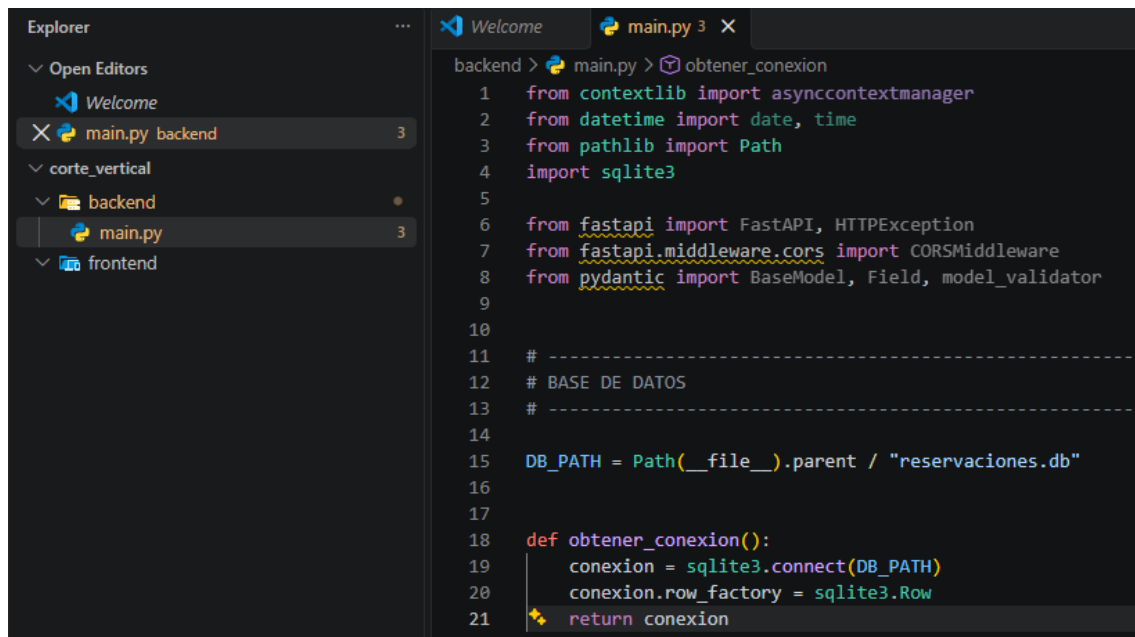
Response esperado:

```
{
  "id": 1,
  "laboratorio": "Laboratorio de Redes",
  "fecha": "2026-09-15",
  "hora_inicio": "10:00:00",
  "hora_fin": "12:00:00",
  "motivo": "Práctica de redes",
  "estado": "pendiente"
}
```

Código HTTP esperado: 201 Created.

Código del backend

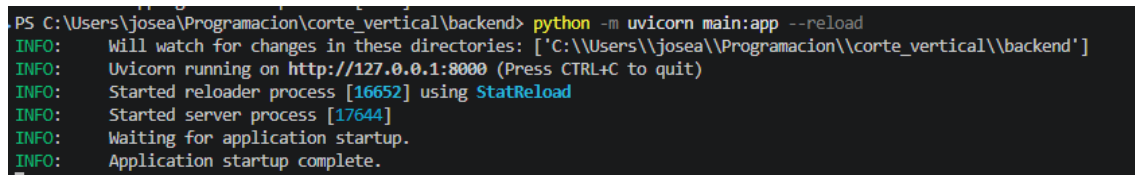
Se implementó la API REST con FastAPI y SQLite. El backend recibe los datos de la reservación, los valida, comprueba si existe un conflicto de horario y, si la operación es válida, almacena la información en la base de datos.



The screenshot shows the Visual Studio Code interface. On the left, the Explorer pane displays the project structure: 'Open Editors' with 'Welcome' and 'main.py backend'; 'corte_vertical' folder containing 'backend' and 'frontend' subfolders, with 'main.py' highlighted under 'backend'. The main editor shows the 'main.py' file with the following Python code:

```
backend > main.py > obtener_conexion
1 from contextlib import asynccontextmanager
2 from datetime import date, time
3 from pathlib import Path
4 import sqlite3
5
6 from fastapi import FastAPI, HTTPException
7 from fastapi.middleware.cors import CORSMiddleware
8 from pydantic import BaseModel, Field, model_validator
9
10
11 # -----
12 # BASE DE DATOS
13 # -----
14
15 DB_PATH = Path(__file__).parent / "reservaciones.db"
16
17
18 def obtener_conexion():
19     conexion = sqlite3.connect(DB_PATH)
20     conexion.row_factory = sqlite3.Row
21     return conexion
```

Probar backend en Swagger



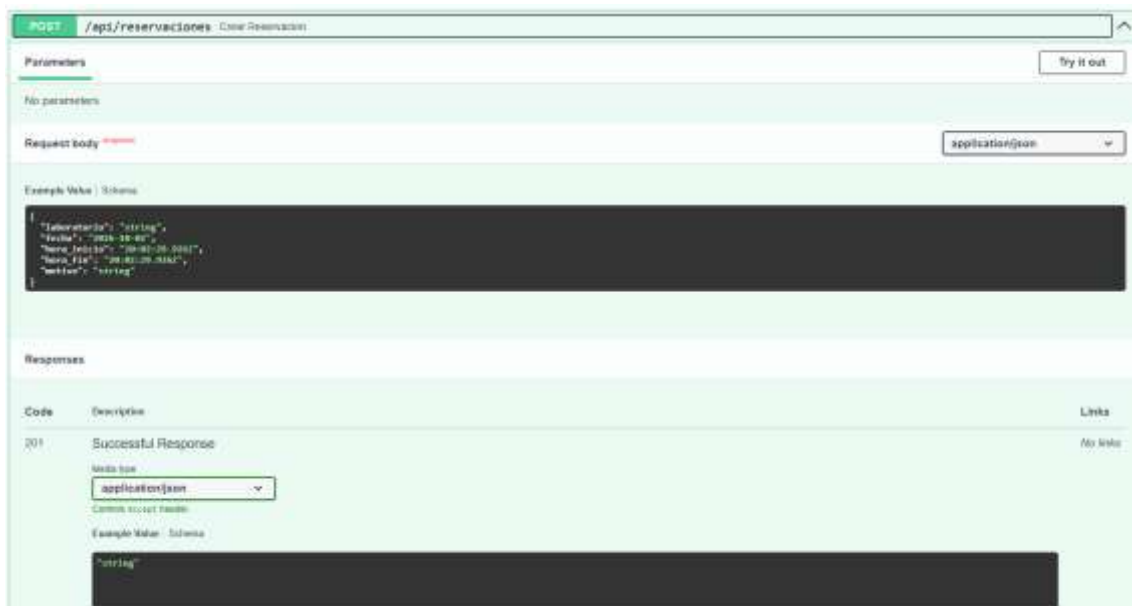
The terminal shows the command to run the backend using uvicorn:

```
PS C:\Users\josea\Programacion\corte_vertical\backend> python -m uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['C:\Users\josea\Programacion\corte_vertical\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [16652] using StatReload
INFO: Started server process [17644]
INFO: Waiting for application startup.
INFO: Application startup complete.
```



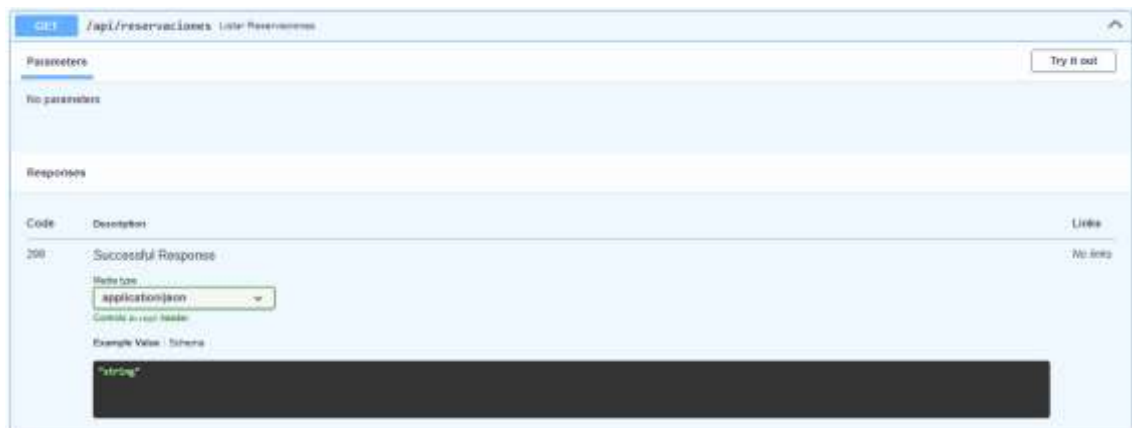
Pruebas del backend

Prueba de registro correcto



La reservación fue registrada correctamente y la API respondió con 201 Created.

Comprobar persistencia



La petición GET `/api/reservaciones` permitió comprobar que la reservación permanece almacenada en SQLite.

Prueba de validación 422

Edit Value | Schema

```
{
  "laboratorio": "Laboratorio de Redes",
  "fecha": "2026-09-15",
  "hora_inicio": "14:00",
  "hora_fin": "12:00",
  "motivo": "Práctica"
}
```

Send

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/api/reservaciones' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "laboratorio": "Laboratorio de Redes",
    "fecha": "2026-09-15",
    "hora_inicio": "14:00",
    "hora_fin": "12:00",
    "motivo": "Práctica"
  }'
```

Code | Details

422 Error: Unprocessable Entity

Response body

```
{
  "detail": {
    "type": "value_error",
    "loc": [
      "hora_fin"
    ],
    "msg": "Value error, la hora final debe ser posterior a la hora inicial.",
    "input": {
      "laboratorio": "Laboratorio de Redes",
      "fecha": "2026-09-15",
      "hora_inicio": "14:00",
      "hora_fin": "12:00",
      "motivo": "Práctica"
    },
    "ctx": {
      "error": []
    }
  }
}
```

Download

Response headers

```
access-control-allow-credentials: true
content-length: 232
content-type: application/json
date: Fri, 05 Oct 2026 13:53:07 GMT
server: waitress
```

La solicitud fue rechazada porque la hora final es anterior a la hora inicial, por lo que no supera la validación del modelo.

Prueba de conflicto 409



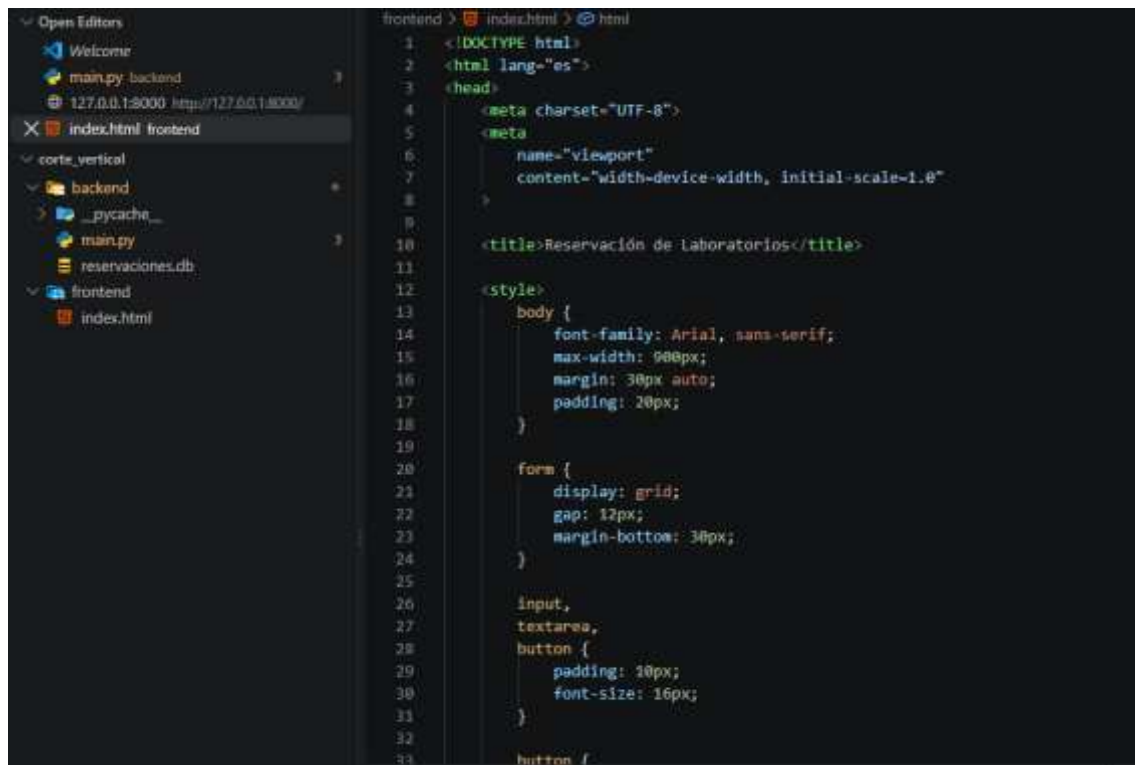
The screenshot shows a REST client interface with a light green header. The status bar at the top indicates a 409 status code and the message "Error: Conflict". Below this, the "Response body" is displayed in a dark box with a "Download" button. The response body contains a JSON object with a "detail" field. Below the response body, the "Response headers" are listed in a dark box.

```
409 Error: Conflict
Response body
{
  "detail": "El laboratorio ya tiene una reservaci3n en ese horario."
}
Response headers
access-control-allow-credentials: true
content-length: 69
content-type: application/json
date: 01,01 Oct 2018 16:07:00 GMT
server: nginx
```

La API detect3 un traslape con una reservaci3n existente y rechaz3 la operaci3n mediante 409 Conflict.

Crear cliente web

Se implementó un cliente web con HTML y JavaScript. El formulario captura los datos de la reservación y utiliza `fetch()` para enviarlos mediante HTTP a la API.



```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="UTF-8">
5   <meta
6     name="viewport"
7     content="width=device-width, initial-scale=1.0"
8   >
9
10  <title>Reservación de laboratorios</title>
11
12  <style>
13    body {
14      font-family: Arial, sans-serif;
15      max-width: 900px;
16      margin: 30px auto;
17      padding: 20px;
18    }
19
20    form {
21      display: grid;
22      gap: 12px;
23      margin-bottom: 30px;
24    }
25
26    input,
27    textarea,
28    button {
29      padding: 10px;
30      font-size: 16px;
31    }
32
33    button {
```

```
const API =
  "http://127.0.0.1:8000/api/reservaciones";
```

```
const respuesta =
  await fetch(
    API,
    {
      method: "POST",
      headers: {
        "Content-Type":
          "application/json"
      },
      body:
        JSON.stringify(datos)
    }
  );
```

Ejecutar la pagina

```
PS C:\Users\josea\Programacion\corte_vertical> cd frontend
PS C:\Users\josea\Programacion\corte_vertical\frontend>python -m http.server 5500
Serving HTTP on :: port 5500 (http://[::]:5500/) ...
::ffff:127.0.0.1 - - [02/Oct/2026 14:18:17] "GET / HTTP/1.1" 200 -
::ffff:127.0.0.1 - - [02/Oct/2026 14:18:17] code 404, message File not found
::ffff:127.0.0.1 - - [02/Oct/2026 14:18:17] "GET /favicon.ico HTTP/1.1" 404 -
```

Reservación de Laboratorios

Corte vertical mínimo: Cliente → API → Base de datos → Respuesta

Laboratorio

Laboratorio de Redes

Fecha

dd/mm/aaaa

Hora inicial

--:--:--

Hora final

--:--:--

Motivo

Describe el motivo de la reservación

Registrar reservación

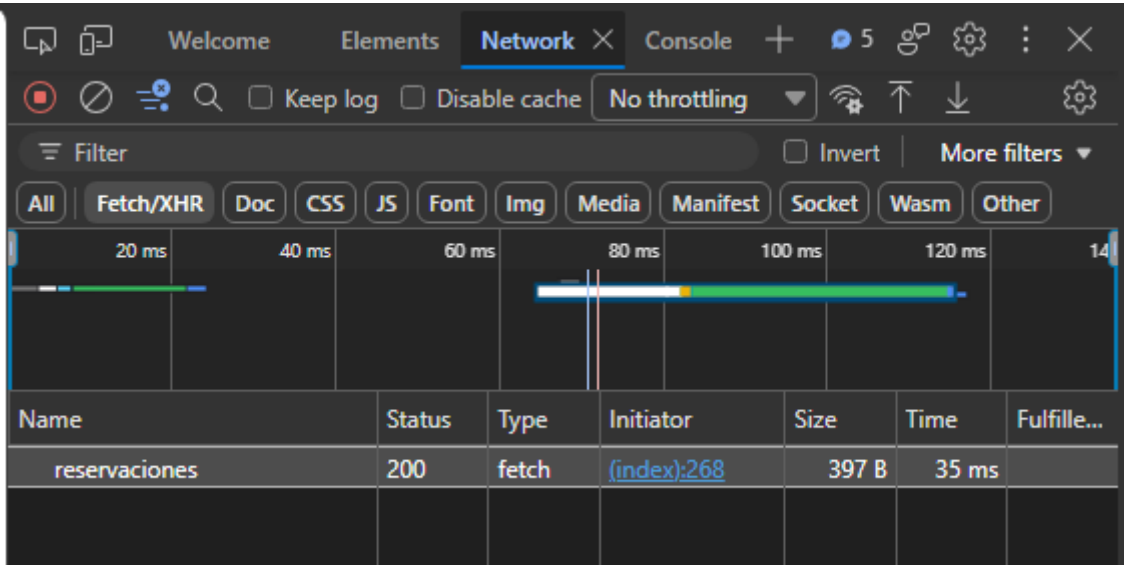
Esperando operación...

Reservaciones registradas

ID	Laboratorio	Fecha	Horario	Estado
1	Laboratorio de Redes	2026-10-10	11:00:00 - 13:00:00	pendiente

El cliente web se comunicó correctamente con la API y mostró el resultado de la reservación creada.

Network



Mediante las herramientas de desarrollo del navegador se inspeccion\u00f3 la petici\u00f3n generada por el cliente web, comprobando el m\u00e9todo POST, la URL del endpoint, el payload enviado y la respuesta 201 Created.

Reservaci\u00f3n de Laboratorios

Cada vertical m\u00f3dulo: Cliente -> API -> Base de datos -> Respuesta

Laboratorio:

Fecha:

Hora inicio:

Hora final:

Motivo:

Error: El laboratorio ya tiene una reservaci\u00f3n en ese horario

Reservaciones registradas

ID	Laboratorio	Fecha	Horario	Estado
1	Laboratorio de Redes	30/08/2026	01:00:00 - 03:00:00	pendiente



Al enviar un horario en conflicto, Network permiti\u00f3 observar la respuesta 409 Conflict generada por la regla de negocio.

Estructura de la base de datos

Campo	Descripción
id	Identificador autoincremental
laboratorio	Nombre del laboratorio
fecha	Fecha de la reservación
hora_inicio	Hora de inicio
hora_fin	Hora final
motivo	Motivo de la reservación
estado	Estado inicial pendiente

Cuestionario de Analisis

1. **¿Dónde está el cliente?** El cliente se encuentra en el archivo frontend/index.html, donde está la interfaz web y el código JavaScript que envía las solicitudes a la API.
2. **¿Dónde está el servidor?** El servidor está en el archivo backend/main.py y se ejecuta mediante FastAPI y Uvicorn en <http://127.0.0.1:8000>.
3. **¿Cuál es el endpoint?** El endpoint principal utilizado es: POST /api/reservaciones. También se utiliza GET /api/reservaciones para consultar las reservaciones registradas.
4. **¿Qué método HTTP utiliza?** Se utiliza POST para registrar una reservación y GET para consultar las reservaciones existentes.
5. **¿Qué información envía el cliente y en qué formato?** Envía el laboratorio, fecha, hora inicial, hora final y motivo en formato JSON.
6. **¿Dónde se validan los datos?** Los datos se validan en el backend mediante el modelo ReservacionEntrada de Pydantic.
7. **¿Por qué no basta validar en HTML?** Porque las validaciones del navegador pueden ser evitadas. El servidor debe validar nuevamente los datos antes de procesarlos y almacenarlos.
8. **¿Qué regla de negocio implementaron?** Se impide registrar una reservación cuando existe otra para el mismo laboratorio y fecha cuyo horario se traslapa.
9. **¿Dónde se persiste la información?** En una base de datos SQLite llamada reservaciones.db, dentro de la tabla reservaciones.
10. **¿Qué significa 201 Created?** Significa que la solicitud fue procesada correctamente y que se creó una nueva reservación.
11. **¿Cuándo utilizan 409?** Se utiliza 409 Conflict cuando se intenta registrar una reservación que entra en conflicto con otra reservación existente.
12. **¿Puede Postman consumir la misma API?** Sí. Postman puede enviar solicitudes HTTP al mismo endpoint utilizando el mismo formato JSON.
13. **¿Puede Flutter consumirla?** Sí. Una aplicación desarrollada en Flutter puede realizar solicitudes HTTP a la misma API.
14. **¿Qué tendría que cambiar para sustituir HTML por Flutter?** Principalmente cambiaría el cliente. El backend, los endpoints, las reglas de negocio y la base de datos pueden mantenerse.
15. **¿Qué sucede si la BD deja de funcionar?** El backend no podría guardar ni consultar las reservaciones y la operación produciría un error.
16. **¿Qué elemento representa el contrato API?** El contrato está representado por el endpoint, el método HTTP, la estructura del JSON enviado y la estructura de la respuesta esperada.

Conclusiones del Laboratorio

Durante el desarrollo de esta práctica se logró implementar un corte vertical completo para el registro de reservaciones de laboratorio, integrando los principales componentes de una arquitectura cliente-servidor. La interfaz desarrollada con HTML permitió capturar los datos del usuario, mientras que JavaScript funcionó como cliente HTTP utilizando `fetch()` para comunicarse con el servicio desarrollado en FastAPI. El backend se encargó del procesamiento de las solicitudes, Pydantic realizó la validación de los datos y SQLite permitió almacenar de forma persistente las reservaciones realizadas. Esta separación de responsabilidades corresponde al modelo planteado en la práctica, donde HTML se encarga de la presentación, JavaScript de la comunicación, FastAPI del servicio, Pydantic de la validación y SQLite de la persistencia.

También se comprobó el correcto funcionamiento de la API mediante diferentes escenarios. Una reservación válida produjo una respuesta 201 Created, mientras que una solicitud con información incorrecta generó 422 Unprocessable Entity. Por otra parte, cuando se intentó registrar una reservación cuyo horario entraba en conflicto con otra existente, el servidor respondió con 409 Conflict, demostrando la aplicación de una regla de negocio dentro del backend. La práctica señala precisamente que el código 422 representa un fallo de validación, mientras que 409 representa una solicitud formalmente válida que entra en conflicto con el estado actual del sistema.

Act 5. Defender y evolucionar una decisión arquitectónica mediante ADR

Objetivo de la práctica

El objetivo de esta práctica es analizar, documentar y defender una decisión arquitectónica relacionada con el Sistema de Reservación de Laboratorios, utilizando un Architecture Decision Record (ADR) como herramienta para registrar de manera estructurada el contexto, los requisitos, las alternativas evaluadas, la decisión tomada y las consecuencias que pueden surgir a partir de ella. La finalidad es comprender que una decisión arquitectónica no debe basarse únicamente en preferencias por una tecnología, sino en argumentos técnicos relacionados con las necesidades reales del sistema, sus restricciones y los requisitos funcionales y no funcionales definidos para el proyecto.

Posteriormente, la práctica tiene como objetivo demostrar que una arquitectura puede necesitar evolucionar cuando cambian los requisitos del sistema. Para ello se introduce un nuevo requisito no funcional relacionado con la actualización automática de la disponibilidad en menos de dos segundos y sin recargar la página. Este cambio obliga a analizar nuevamente la arquitectura, modificar la importancia de ciertos criterios, reevaluar las alternativas y determinar si la decisión anterior debe mantenerse, modificarse o sustituirse parcialmente.

Finalmente, mediante la creación del ADR-002, se busca comprender la importancia de conservar la trazabilidad de las decisiones arquitectónicas. En lugar de eliminar la decisión anterior, se mantiene el historial y se documenta la evolución hacia una solución que conserva REST para las operaciones CRUD e incorpora SSE para las actualizaciones servidor-cliente. Con ello, la práctica permite entender que las decisiones arquitectónicas dependen del contexto y que los ADR funcionan como una memoria técnica que explica qué se decidió, por qué se decidió y qué requisito provocó posteriormente su evolución.

Descripción del escenario

El caso de estudio corresponde a un Sistema de Reservación de Laboratorios. El sistema permite consultar laboratorios y disponibilidad, crear y cancelar reservaciones, aprobar solicitudes y consultar el historial. En la arquitectura inicial, la comunicación entre el cliente y el servidor se realiza mediante REST con JSON.

Requisitos funcionales

ID	Requisito funcional
RF01	Consultar laboratorios.
RF02	Consultar disponibilidad.
RF03	Crear reservación.
RF04	Cancelar reservación.
RF05	Aprobar reservación.
RF06	Consultar historial.

Requisitos no funcionales iniciales

ID	Requisito no funcional
RNF01	La solución deberá funcionar desde aplicaciones web y móviles.
RNF02	La API deberá ser sencilla de consumir desde diferentes tecnologías.
RNF03	El sistema deberá favorecer facilidad de mantenimiento.
RNF04	La respuesta de las operaciones CRUD deberá ser inferior a 3 segundos.
RNF05	El equipo dispone de experiencia básica en HTTP y APIs REST.
RNF06	No se requiere inicialmente actualización automática en tiempo real.

Pregunta arquitectónica

¿Qué mecanismo de comunicación utilizaremos inicialmente entre las aplicaciones cliente y el backend para las operaciones principales del Sistema de Reservación de Laboratorios?

Alternativas y matriz inicial

Tecnología	Puntuación
REST	4.70
gRPC	3.35
WebSocket	3.25
SSE	3.45

De acuerdo con la matriz inicial, REST obtiene la puntuación más alta debido a que los criterios con mayor peso favorecen las operaciones CRUD, la compatibilidad, la facilidad de implementación y el mantenimiento.

ADR-001

ADR-001 – REST como mecanismo principal de comunicación

Estado: Aceptado.

El sistema será utilizado desde clientes web y móviles. Las operaciones iniciales corresponden principalmente a consultas, creación, actualización y cancelación de información. En esta etapa no se requiere comunicación bidireccional permanente ni actualización automática en tiempo real.

Alternativas consideradas:

REST, gRPC, WebSocket y Server-Sent Events (SSE).

Decisión:

Utilizar REST sobre HTTP/HTTPS con JSON como mecanismo principal de comunicación entre las aplicaciones cliente y la API.

Justificación:

REST se adapta adecuadamente a las operaciones CRUD del sistema, ofrece compatibilidad con aplicaciones web y móviles, facilita las pruebas y el mantenimiento y aprovecha la experiencia previa del equipo con HTTP y APIs REST.

Consecuencias positivas:

Implementación sencilla, interoperabilidad, documentación mediante OpenAPI, facilidad de depuración y menor curva de aprendizaje.

Consecuencias negativas:

No proporciona comunicación bidireccional continua y las actualizaciones frecuentes podrían requerir polling.

Riesgos:

La aparición futura de requisitos de tiempo real, streaming o un exceso de solicitudes mediante polling.

Evidencia:

Matriz de decisión REST = 4.70/5, prueba POST /api/reservaciones, pruebas con navegador o cliente API y modelo C4.

Defensa del ADR-001

¿Por qué REST?

Porque el sistema realiza principalmente operaciones CRUD y REST ofrece buena compatibilidad, facilidad de implementación y mantenimiento.

¿Por qué no WebSocket?

Porque en el escenario inicial no se requiere comunicación bidireccional permanente ni actualización en tiempo real.

¿Por qué no gRPC?

Porque REST resulta más sencillo de consumir desde clientes web y móviles y el equipo ya tiene experiencia con HTTP y APIs REST.

¿Por qué JSON?

Porque es un formato ampliamente utilizado, sencillo de interpretar y compatible con múltiples tecnologías.

¿Qué desventaja tiene REST?

No ofrece actualizaciones automáticas en tiempo real por sí solo.

Nuevo requisito RNF07

RNF07: Cuando una reservación sea creada, aprobada, cancelada o modificada, los usuarios que visualicen la disponibilidad deberán observar el cambio automáticamente en menos de 2 segundos y sin recargar la página.

Análisis de impacto

El RNF07 cambia las necesidades del sistema porque ahora se requiere actualización automática en tiempo real desde el servidor hacia los clientes. REST continúa siendo adecuado para las operaciones CRUD, pero por sí solo no proporciona el mecanismo de actualización automática requerido.

Por esta razón, es necesario reevaluar las alternativas sin eliminar automáticamente la arquitectura existente.

Nueva evaluación

Tecnología	Nueva puntuación
REST	3.70
WebSocket	4.00
SSE	4.15

Con el nuevo requisito RNF07, el criterio de tiempo real adquiere mayor importancia. SSE obtiene la puntuación más alta con 4.15/5, mientras que REST mantiene su utilidad para las operaciones CRUD.

Nueva decisión

Se decide mantener REST para las operaciones transaccionales del sistema e incorporar Server-Sent Events (SSE) para enviar automáticamente actualizaciones de disponibilidad desde el servidor hacia los clientes conectados.

ADR-002

ADR-002 – Incorporar Server-Sent Events para actualizaciones de disponibilidad

Estado: Aceptado.

El RNF07 exige que los cambios en las reservaciones sean visibles automáticamente en menos de 2 segundos y sin que el usuario tenga que recargar la página.

Alternativas consideradas:

REST + polling, sustituir REST por WebSocket, REST + WebSocket y REST + SSE.

Decisión:

Mantener REST para las operaciones CRUD e incorporar SSE para la comunicación servidor → cliente.

Consecuencias positivas:

Permite actualizaciones automáticas, conserva la arquitectura REST existente y reduce el uso de polling.

Consecuencias negativas:

Requiere mantener conexiones abiertas, introduce un segundo mecanismo de comunicación y aumenta la complejidad del sistema.

Estado actualizado del ADR-001

ADR-001: Sustituido parcialmente por ADR-002.

REST continúa utilizándose para las operaciones CRUD, mientras que SSE se incorpora específicamente para satisfacer el requisito de actualización automática.

Diagrama de arquitectura antes

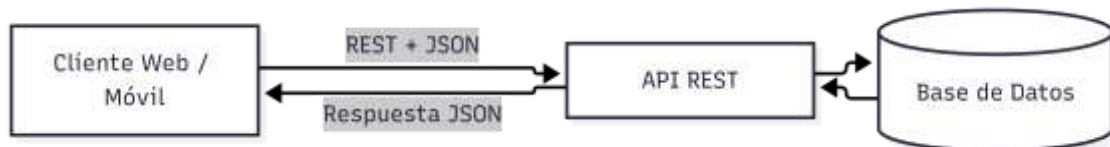
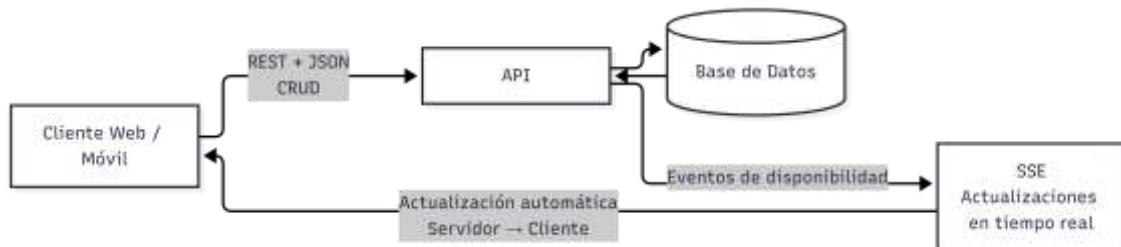


Diagrama de arquitectura después



La diferencia importante es que ahora aparece **SSE del servidor hacia el cliente**.

Cuestionario de Analisis

1. **¿Por qué no existe una arquitectura perfecta?** Porque cada arquitectura depende de requisitos, restricciones y contexto específicos.
2. **¿Qué diferencia existe entre una decisión y una preferencia?** Una decisión se justifica mediante requisitos y evidencia; una preferencia se basa principalmente en gustos personales.
3. **¿Qué función cumplen los RNF?** Definen características como rendimiento, mantenimiento, disponibilidad, seguridad o tiempo real.
4. **¿Por qué una decisión correcta puede necesitar cambiar?** Porque los requisitos o el contexto del sistema pueden evolucionar.
5. **¿Por qué no debemos borrar ADR antiguos?** Porque conservan la historia y las razones de las decisiones anteriores.
6. **¿Qué significa que una arquitectura evolucione?** Que se modifica para adaptarse a nuevas necesidades sin perder la trazabilidad.
7. **¿Qué consecuencias negativas tuvo la primera decisión?** REST no proporcionaba actualización automática en tiempo real.
8. **¿Qué requisito provocó el cambio?** RNF07.
9. **¿Qué habría ocurrido si no modificaban la arquitectura?** No se cumpliría correctamente el requisito de actualización automática en menos de 2 segundos.
10. **¿La nueva solución es más compleja?** Sí, porque utiliza REST y SSE.
11. **¿La complejidad adicional está justificada?** Sí, porque permite satisfacer el nuevo requisito manteniendo REST para las operaciones CRUD.
12. **¿Qué otro requisito podría volver a cambiar la arquitectura?** Por ejemplo, requerir comunicación bidireccional en tiempo real entre los usuarios.

Conclusión

En esta práctica se documentó y analizó la evolución de una decisión arquitectónica mediante el uso de ADR. Inicialmente se seleccionó REST con JSON como mecanismo principal de comunicación debido a que el Sistema de Reservación de Laboratorios estaba orientado principalmente a operaciones CRUD y no requería actualizaciones automáticas en tiempo real.

Posteriormente, la aparición del RNF07 modificó el contexto del sistema al exigir que los cambios de disponibilidad fueran visibles automáticamente en menos de 2 segundos. Esto hizo necesario reevaluar las alternativas y condujo a la incorporación de SSE, manteniendo REST para las operaciones transaccionales.

La práctica permitió comprender que una decisión arquitectónica no es permanente ni debe considerarse correcta para todos los escenarios. Los ADR permiten conservar las razones, consecuencias y evidencia de cada decisión, facilitando la evolución de la arquitectura sin perder la trazabilidad de los cambios realizados.